A

如果答案是 Yes，那么我们实际上可以使用至多一次操作使得序列非递减。

从头开始遍历，遇到第一个递减的位置时，我们一定需要移动当前的前缀。因此判断移动该前缀之后是否非递减即可，

B

考虑问题的简化版本：判断序列中是否存在一个数字 x 能够整除序列中的所有数字。

对于简化版本，如果如果数字存在，那么一定是序列中的最小值，简单判断即可。

对于该问题，首先先移除能够被最小值整除的数字，之后就得到了问题简化版本，再次判断即可。

C

假设最终答案为 x，不难证明我们只需要至多一次操作就可以使得一个长度至少为 2 的子数组的所有元素都变成 x ，那么一定存在一个长度为 3 且中位数为 y（$y \geq x$）的子数组。

如果 $n = 2$，则答案是最小的元素。

如果 $n \geq 3$，我们遍历所有长度为 3 的子数组，答案是所有长度为 3 的子数组的中位数的最大值。

D

不难发现，当第一次有一个点被染成蓝色后，在之后的移动中，可以忽略染成红色，再染成蓝色这个过程。

我们称第一个被染成蓝色的点为 r。那么不难发现，P_A 比 P_B 提前到达了这个节点，考虑后续 P_B 的所有操作，P_A 可以在到达 r 后一个一个复原这些操作，那么后续 P_B 走过的所有节点就已经被染成红色过了。

那么假设我们知道了 r 这将是一个经典问题，假设树上距离 r 最远的点和 r 的距离是 d，那么答案就是 $2(n-1)-d$。那么我们此时考虑 P_A 和 P_B 的策略，两者一定是互相靠拢，然后直到染蓝第一个点为止。因为考虑如果到其他点，虽然 d 的值可能增大，但是 d 的值每增大 1，P_A 和 P_B 相遇的时间也会至少增加 1，因此答案不会更优。

E

我们可以在 $O(1)$ 时间复杂度下维护 $0/1/2/3+$ 个黑色子节点的黑色节点数量，因为当我们翻转一个节点的颜色时，只有该节点及其父亲节点的状态会受到影响。

不难发现如果黑色节点形成一条链：

- 不存在 $3+$ 个黑色子节点的黑色节点，并且至多只有一个黑色节点有两个黑色子节点。
- 至多只有一个父节点为白色的黑色节点。
- 如果一个黑色节点有两个黑色子节点，则其父节点必须是白色的。

根据条件即可在线性时间内判断。

F

考虑从 0 到 $n-1$ 枚举每个数字是否包含在 S 中，当我们枚举完前 x 个数字时，本质上只剩余 2^{n-x} 个约束。

假设当前枚举达到i，对于剩下的约束，假设T_1 和 T_2 是两个集合，它们之间唯一的区别是是否包含 i（T_1 包含 i）：

- S 包含 i。 我们可以将 T_1 和 T_2 合并为新约束 T'，并且有 $v_{T'} = (v_{T_1} >> 1) \& v_{T_2}$。
- S 不包含 i。 我们可以将 T_1 和 T_2 合并为新约束 T'，并且有 $v_{T'} = v_{T_1} \& v_{T_2}$。

时间复杂度为$O(n \cdot 2^n)$。

G.

Hint.1

如果 s, t 同时带有 $*$，或同时不带有 $*$，那么者将会是一个简单的问题，你可以尝试解决这个问题吗?

Hint.2

根据 **Hint.1**，我们讨论掉了两种情况，在剩余的情况中，不失一般性的，我们认为只有 t 有 $*$。假设我们将 t 写成 $t'_0 * t'_1 * t'_2 * \cdots * t'_k$。其中 t'_i 不带 $*$，假设对于所有 i，满足 $|t'_i| \le 50$，你可以轻松解决这个问题吗?

Hint.3

有一个严格为该问题子问题的问题，即给定两个只带 a 到 z 和 $-$ 的 s，t，你需要找到 t 在 s 中所有可以匹配的位置。

该问题有一个经典的做法，将 $-$ 设为 0，a 到 z 依次为 1 到 26，然后令 $f_i = \sum_{j=1}^{m}[s_{i+j-1} > 0] \times [t_j > 0](s_{i+j-1} - t_j)^2$，那么所有 $f_i = 0$ 的位置就满足 $s[i, i+m-1]$ 可以和 t 匹配。

将 f 贡献拆开，$f_i = \sum_{j=1}^{m} s_{i+j-1}^2 + s_j^2 - 2\sum_{j=1}^{m} s_{i+j-1}t_j - \sum_{j=1}^{m}[s_{i+j-1} = 0]t_j^2 - \sum_{j=1}^{m} s_{i+j-1}^2[t_j = 1]$。对于 $\sum_{j=1}^{m} s_{i+j-1}^2 + s_j^2$ 可以前缀和出处理，对于剩余部分可以用快速傅里叶变在 $O(n\log n)$ 内解决。

这个算法在原问题中有什么用呢?

Editorial

在下文中，我们认为 n, m 同阶。

首先是 s, t 都不带 $*$ 的情况，只需要按位比较过去即可。

考虑 s, t 都带 $*$ 的情况，如果 s 的开头不是 $*$ 或 t 的开头不是 $*$：

- 如果 s 和 t 的开头有之一是 $*$ 或开头可以匹配，则删除两串非 $*$ 的开头。
- 如果不可以匹配，显然答案是 `No` 。

队末尾进行同样的操作，不难发现会归约到 $*s$' 和 $t'*$ 的两个串，其中 s' 和 t' 为任意字符串。那么不难发现此时 $t's'$ 就同时和两个串形成匹配。因此只要能成功删除首尾，答案此时一定是 `Yes` 。

接下来是最难的情况，不失一般性的，我们假设只有 t 有 $*$，否则可以交换 s, t。我们仍然先将首尾删除。不难发现此后 t 可以写成 $*t'_1 * t'_2 * \cdots * t'_k*$。

那么此时我们就会有一个贪心，我们从 1 到 n 遍历 i，每次找出 t'_i 在 s 中第一个可以匹配的位置，删除 s 中这些匹配的字符和前面的所有字符，即删除 s 的一个前缀，然后直到全部匹配完成或发现 s 中没有匹配位置。

假设每次我们都用快速傅里叶变换暴力解决这个问题，具体方案见 **Hint.3**，那么复杂度就是 $O(nk\log n)$。

但不难发现这样做非常浪费，实际上我们可以这样：

- 匹配 t'_i 时，每次只拿出 s 的前 $|2t'_i|$ 个字符，尝试和 t'_i 进行匹配。因为如果匹配成功，那么由于删除了所有和 t'_i 匹配的位置，不难发现至少删除 $|t'_i|$ 个字符。而如果匹配失败，那么不难发现 s

的前 $|t'_i|$ 个字符必然不会再有用，也可以删除。因此我们在 $O(|t'_i|\log|t'_i|)$ 的复杂度内删除了 s 中的至少前 $|t'_i|$ 个字符。

由于 s 总长度是 n，所以总复杂度不超 $O(n\log n)$。

H

Editorial

考虑字符串 s 的最大字符 m。

如果 m 只出现一次，那么显然以它开头的字符串是字典序最大子串。 为了使其字典序最小，m 应该放在最后。 其他字符可以任意排列。

如果 m 出现多于两次，可以证明答案字符串的开头和结尾必须是 m。如果字符串的开头不是 m，将开头的字符移动到某个非第一个 m 之前，可以使以这个 m 开头的后缀字典序更小，而以这个 m 之后的 m 开头的后缀字典序保持不变。对于不以 m 开头的后缀，它们肯定不是最大的，所以不需要考虑。如果字符串的结尾不是 m，首先将其移除，所有后缀的字典序都会变小。同样，也可以将其放置在某个 m 前面以减少字典序。

在这种情况下，答案总是以 $ms_1ms_2m\ldots ms_tm$ 的形式出现，其中 $s_1,s_2,\ldots,s_t$ 包含比 m 小的字符，也可以是空字符串。现在假设集合 $\{s_1,s_2,\ldots,s_t\}$ 已经确定，考虑如何排列它们的顺序。假设我们目前在比较两个后缀 $S_i=ms_im\ldots ms_tm$ 和 $S_j=ms_jm\ldots ms_tm$，并且假设 s_{i+k} 和 s_{j+k} 是第一个不同的两个子串。那么 $ms_im\ldots ms_{i+k-1}$ 和 $ms_jm\ldots ms_{j+k-1}$ 必须相等，并且 S_i 和 S_j 之间的大小关系与 ms_{i+k} 和 ms_{j+k} 之间的大小关系相同。通过这种方式，所有的 ms_i 可以按字典序排列，并视为一个新字符，然后解决 $(ms_1)(ms_2)\ldots(ms_t)$ 的问题即可。

现在考虑如何确定集合 $\{s_1,s_2,\ldots,s_t\}$。对于某个 s_i，它显然应该是有序的，因为这可以使 ms_i 更小。还有另一个性质。假设存在 $s_i=p_ic$ 和 s_j 使得 ms_j 是当前字典序最大的字符串，并且 $mp_i<ms_j$，其中 c 是一个小于 m 的字符。将 c 放在 s_j 之后可以使答案更小。

该性质的证明如下。为了不失一般性，设 $f_1=p_ic$，$f_2=p_i$，$f_3=s_jc$，$f_4=s_j$，则我们有 $mf_2<mf_1<mf_4$ 且 $f_3m<f_4m$。考虑迭代后的子问题，根据定义，f_4 是问题中最大的字符。如果使用 f_1 和 f_4，并且在最优解字符串中某个 f_1 的左侧有一个 f_4，那么将该 f_1 和其左侧最近的 f_4 分别替换为 f_2 和 f_3，可以很容易地证明最大的后缀一定会变得更小或保持不变。否则，所有的 f_1 都在所有 f_4 的左侧，此时可以任意地将一些 f_1 和 f_4 分别替换为 f_2 和 f_3，也可以很容易地证明最大的后缀仍然变得更小或保持不变。

根据上述讨论，我们现在可以给出该问题的最终贪心算法。假设最大的字符 m 出现了 $t+1$ 次。如果 $t=0$，那么将唯一的 m 放在最后，其他字符可以任意排列。否则，假设有 p 个字符小于 m，如果

$p \le t$，可以证明在任意两个 m 之间最多只有一个字符。考虑 $|s_i| \ge 2$，$s_j = \varepsilon$（空字符串），那么它们满足上述性质，并且将 s_i 的最后一个字符交换到 s_j 更优。因此，$s_1, \ldots, s_p$ 是单个字符，$s_{p+1} = \cdots = s_t = \varepsilon$，然后递归地解决子问题。注意，为了保证时间复杂度，当 $t \gg p$ 时，需要结合多轮递归。

如果 $p > t$，同样可以证明不会存在一个 s_i 是空字符串。此外，考虑所有 s_i 的第一个字符，对于这些 t 个字符中不是最大的那些，利用相同的性质，可以证明它们所在的字符串的长度必须为 1，并且后面不会跟随其他字符。另外，由于 s_i 是有序的，可以证明所有 s_i 的第一个字符必须是小于 m 的最小的 t 个字符。接下来，剩下的字符将仅填充在这些 t 个字符中的最大字符之后，同样可以证明，这些字符串的第二个位置填充的也是剩余字符中最小的，依此类推，直到所有字符都用完。

注意，每次递归的复杂度是线性的，并且每次递归至少使字符数减半，因此总复杂度是 $\mathcal{O}(n \log n)$。